Architecture Guide

1. Purpose

This document explains how the project is structured, how the components fit together, and how the dashboard works end to e

It is intended for:

- developers who want to understand the codebase,
- researchers who want to describe the system in a paper,
- maintainers who need to extend or debug the workflow,
- reviewers who want a precise technical overview.

2. System Architecture at a Glance

The project is organized into four layers:

1. Browser frontend.
2. Flask backend server.
3. Core screening and storage package.
4. Supporting assets, tools, and research files.

The browser handles user interaction. The backend handles validation, scoring, persistence, and PDF export. The Python pacl

3. Frontend Architecture

3.1 Frontend Files

- web/index.html (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/index.html)
- web/app.js (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/app.js)
- web/styles.css (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/styles.css)

3.2 Frontend Responsibilities

The frontend is responsible for:

- rendering the assessment and analytics UI,
- switching the interface language,
- capturing questionnaire and narrative input,
- managing audio, image, and passive biomarker inputs,
- showing live previews before save,
- displaying saved results and reports,
- hiding demo records from the visible user list,
- supporting report hiding and record deletion actions,

- rendering offline and sync status.

3.3 Frontend State

The dashboard uses local state to track:

- the current language,
- the selected record,
- all loaded assessment results,
- filtered results,
- draft preview data,
- hidden report IDs,
- offline sync state,
- typing events and media capture state,
- quality-check results,
- model-statistics state.

This state is then used to render the visible panels.

3.4 Frontend Panels

Assessment Workspace

This panel is the intake form.

It includes:

- profile fields,
- consent,
- questionnaire items,
- narrative input,
- optional upload fields,
- live prediction preview,
- capture and reset actions.

Adaptive Test

This panel presents one question at a time and adjusts the next question based on the current response state.

Analytics Hub

This panel is the detailed analysis view.

It shows:

- score comparison,
- multimodal contribution,
- NLP and safety signals,
- modality quality,
- recommendation and disclaimer,
- model statistics,
- quality-check results,
- trajectory and trend summary.

Records and Reports

This panel is for browsing, inspecting, exporting, and removing records.

It includes:

- searchable record list,
- detailed assessment view,
- PDF export,
- record delete action,
- report hide action.

4. Backend Architecture

4.1 Backend Entry Point

File:

- dashboard_server.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/dashboard_server.py)

The backend is a Flask application that serves both the static dashboard and the API.

4.2 Backend Responsibilities

The backend performs these core tasks:

- validates assessment submissions,
- normalizes profile and questionnaire fields,
- computes preview and saved-assessment multimodal outputs,
- persists assessment records,
- fetches records by ID,
- deletes records,
- provides trajectory summaries,
- returns model statistics,

- runs saved-assessment quality checks,
- renders PDF reports,
- supports adaptive screening routes,
- exposes sample and database metadata endpoints.

4.3 Backend Validation Path

When the backend receives a screening request, it:

1. extracts profile and questionnaire data,
2. validates consent and required fields,
3. builds or normalizes multimodal inputs,
4. computes screening outputs,
5. persists the record if it is a save operation,
6. returns JSON to the frontend.

If required data is missing, the backend returns a structured error message rather than failing silently.

4.4 Backend Output Types

The backend produces:

- live preview results,
- saved assessment records,
- PDF report bytes,
- quality-check metrics,
- trajectory data,
- adaptive question payloads,
- database and audit metadata.

5. Core Python Package Architecture

5.1 Package Folder

- src/mental_health_screening/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/src/mental_health_screening)

5.2 Module Roles

assessment.py

Defines the questionnaire structure and scoring helpers.

feature_extract.py

Extracts and normalizes text, audio, image, and passive features.

predict.py

Combines questionnaire and modality signals into screening scores and confidence estimates.

storage.py

Manages persisted assessment records, retrieval, deletion, and audit-related helpers.

report.py

Creates assessment PDFs and quality-check PDFs.

model_store.py

Summarizes model bundle metadata and trained-bundle coverage.

training.py

Builds and retrains model bundles for the projects supported modalities.

utils.py

Provides shared utilities used across the package.

5.3 Separation of Concerns

The package is intentionally split so that:

- feature extraction stays separate from storage,
- storage stays separate from reporting,
- training stays separate from runtime inference,
- shared utilities stay reusable across the project.

That separation makes the code easier to maintain and easier to cite in a paper.

6. Data Flow Architecture

6.1 From User Input to Assessment Record

The data flow is:

1. User enters profile and questionnaire data in the browser.

2. User adds narrative text and optional media inputs.
3. Frontend creates a payload.
4. Payload is sent to the backend preview or save route.
5. Backend validates and scores the record.
6. Backend stores the record.
7. Frontend reloads the list and detail views.
8. User can export or fetch the record later.

6.2 Preview Flow

Preview flow is used before saving:

- the frontend builds a temporary payload,
- the backend returns preview scores and notes,
- the dashboard renders the draft view,
- no permanent save is required yet.

6.3 Save Flow

Save flow is the permanent path:

- profile and questionnaire are validated,
- multimodal scores are computed,
- record is stored in the database,
- record ID is returned,
- analytics and records views update.

6.4 Offline Flow

If the backend cannot be reached:

- the dashboard can show an offline preview,
- the assessment can be stored locally,
- the system queues the record for later sync,
- the user sees a clear offline status.

7. Visibility and Record Rules

The project intentionally distinguishes between different kinds of records.

7.1 Visible User Records

These are the records shown in the records table and report panel.

7.2 Demo Records

Demo records are bundled for development, testing, or examples.

They are:

- hidden from the user-facing record list,
- excluded from PDF export,
- not presented as real user data.

7.3 Hidden Reports

A report can be removed from the visible records/report panel without deleting the underlying assessment.

This supports privacy-friendly workflows where a user wants to hide a report from the interface but keep the source record ava

8. Multilingual Architecture

The app supports English, Hindi, and Bengali across the main UI.

The multilingual design applies to:

- labels and headings,
- recommendation text,
- disclaimers,
- validated instruments,
- offline notices,
- quality-check summaries,
- adaptive-screening prompts.

The architecture keeps translation data inside the frontend so the browser can switch instantly without requiring a full page relo

9. Scoring Architecture

The scoring system combines multiple evidence sources.

9.1 Questionnaire Scores

The questionnaire provides domain-level risk information from structured items.

9.2 Text Features

Text analysis contributes:

- sentiment,
- emotion,
- emotion intensity,
- distress phrases,
- safety language,
- agrarian distress markers,
- model-backed NLP features when available.

Current runtime priority:

- text_transformer is the primary scorer when available
- text remains the fallback bundle
- weak text domains are lightly retuned from existing sentiment, emotion, and keyword cues

9.3 Audio Features

Audio can contribute:

- file metadata,
- voice-derived indicators,
- confidence signals,
- heuristic fallback values when full inference is not possible.

Current runtime priority:

- audio is the primary audio scorer
- audio_spectrogram and audio_sequence remain secondary stabilizers
- mood-heavy audio domains are allowed to benefit from the main bundle more than the auxiliary branches

9.4 Image Features

Image inputs contribute:

- file metadata,
- face or vision-related indicators,
- confidence estimates,
- heuristic fallback values when full inference is not possible.

Current runtime priority:

- image_dl is the primary image scorer
- the classical image bundle remains a stabilizer and fallback

9.5 Passive Biomarker Features

Passive signals can include:

- typing rhythm,
- passive video metadata,
- movement or rhythm-derived indicators.

9.6 Fusion Logic

The system fuses the inputs into:

- per-domain scores,
- overall confidence,
- overall risk labels,
- recommendation text,
- modality quality summaries.

The current fusion layer gives:

- text close-following influence behind the strongest visual/acoustic signals
- audio stronger influence in the final blend
- image DL slightly more influence than the classical image path
- comorbidity a blended view of the upstream modality consensus and the chain ensemble

10. Recommendation and Disclaimer Architecture

The recommendation engine uses the final risk distribution to produce the end-user explanation.

It is structured to:

- give a clear next step,
- vary by risk level,
- localize into the active language,
- include a screening disclaimer,
- distinguish screening support from diagnosis.

This is intentionally separate from the scoring logic so the result can be explained in plain language without changing how the s

11. Model Statistics Architecture

The model statistics panel displays:

- loaded model family,
- transformer labels,

- sample counts,
- coverage,
- manifest information,
- dataset root,
- trained versus fallback sources.

This gives the user context about the source of the predictions and helps identify when the dashboard is relying on a trained bu

12. Quality-Check Architecture

The quality-check workflow is part of the analytics layer.

It:

- reads saved assessments,
- computes proxy metrics,
- highlights mismatches,
- can be exported as PDF,
- can be used to monitor dashboard consistency over time.

13. Trajectory and Trend Architecture

The app builds longitudinal views from repeated assessments of the same person when patient identity can be linked.

These views show:

- baseline to current change,
- recent change,
- volatility,
- domain drift,
- overall trajectory status.

This is useful for repeat screening and follow-up workflows.

14. Training and Research Architecture

The project includes supporting research and training flows:

- proxy pretraining from public emotion datasets,
- clinical retraining from approved datasets,
- manifest building,
- federated or multi-center training support,
- saved-assessment evaluation,
- publication-ready paper scaffolding.

This means the repository is not only an app, but also a research and experimentation workspace.

The trained sklearn estimators in models/mental_health_screening/ also have ONNX exports in models/mental_health_screeni

15. Deployment Architecture

The standard local deployment path is:

python dashboard_server.py

Then open:

http://127.0.0.1:8000/

Typical deployment concerns:

- backend availability,
- database persistence,
- service worker behavior,
- offline fallback,
- report generation,
- model bundle presence,
- secure handling of records.

16. Extensibility Notes

The architecture is already set up so new work can be added in clean layers:

- new UI panels can be added in web/,
- new API routes can be added in dashboard_server.py,
- new scoring features can be added in src/mental_health_screening/,
- new experiments can be added to tools/,
- new documentation can be added to docs/,
- new publication material can be added to paper/.

17. How the Current Working Features Fit Together

The important working behaviors in the current codebase are:

- the dashboard only shows visible user records in the main records/report flow,
- demo records are filtered out of user-facing panels,
- report hiding works without destroying the record,
- text and image are treated as usable when real signal exists,

- recommendation wording adjusts to the detected risk level,
- multilingual content is integrated through the frontend translation layer,
- offline preview is available when the backend is unreachable,
- PDFs and quality checks are generated from saved data.

18. Related Documentation

Other documents in the repo:

- docs/PROJECT_DOCUMENTATION.md (/c:/Rural%20Mental%20Heath%20Screening%20AI/docs/PROJECT_DOCUMENT
- paper/manuscript.tex (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/manuscript.tex)
- paper/research-proposals.md (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/research-proposals.md)